ANTEATER POKER
SOFTWARE SPECIFICATIONS
VERSION 1.0

TEAM 21
Naveen Khan
Mia Ness
Austin Nguyen
Queency Tan
Aman Macwan
Gurvin Dhanjal

University of California, Irvine

TABLE OF CONTENTS

Glossary

ANTEATER CARD - NEW CARD ADDED TO THE GAME

BANK - THE AMOUNT OF MONEY A PLAYER HAS

BLIND - AN AMOUNT OF MONEY BET BY A CERTAIN PLAYER. BIG BLIND IS SET TO ONE PLAYER AND SMALL BLIND, WHICH IS HALF A BIG BLIND, IS SET TO THE NEXT PLAYER.

CALL - RAISING YOUR BET TO MATCH THE CURRENT BET

CHECK - MOVE ON WITHOUT ADDING TO THE POT

DEAL - DISTRIBUTE CARDS

DECK - THE CARDS OF THE GAME

FLUSH - 5 CARDS OF THE SAME SUIT

FOLD - QUIT THE ROUND

FOUR OF A KIND - FOUR CARDS OF THE SAME RANK

FULL HOUSE - 3 CARDS OF THE SAME RANK AND TWO CARDS THAT ARE THE SAME OF A DIFFERENT RANK

HAND - CARDS OF THE PLAYER

HIGH CARD - A CARD OF THE MOST VALUE (USUALLY A ROYAL CARD)

PAIRS - TWO OF THE SAME CARDS

POT - THE TOTAL AMOUNT OF MONEY BET BY ALL PLAYERS

RAISE - BET MORE MONEY

RANK - NUMERICAL IMPORTANCE OF EACH CARD (THEIR NUMBER)

ROYAL CARD - ACE, KING, QUEEN, JACK CARD

ROYAL FLUSH - A FLUSH WITH A 10, JACK, QUEEN, KING, AND ACE

SHUFFLE - RANDOMLY CHANGE THE ORDER OF THE CARDS

STRAIGHT - FIVE CONSECUTIVE RANKS IN ORDER

SUIT - HEARTS, DIAMONDS, CLUBS, AND SPADES

THREE OF A KIND - THREE CARDS OF THE SAME RANK

TWO PAIR - A PAIR OF ONE RANK WITH A PAIR OF A DIFFERENT RANK

1. Poker Client Software Architecture Overview

1.1 Client Main Data Types and Structures

The client-side system stores a synchronized copy of the poker game received from the server. The client is responsible for displaying game information, receiving player input, controlling bot players, and sending player actions back to the server. The client does not control the official game state. The server always stores the authoritative version of the game.

Important client-side structures include Card, Player, PlayerAction, GameState, and NetworkPacket.

Card:
Represents a single playing card. Stores the card rank, suit, and whether the card is a normal card or the Anteater card.

Player:
Stores player information visible to the client including player name, bank balance, current bet, fold status, seat number, player type, and hand cards.

PlayerAction:
Represents an action selected by the player or bot. Actions include check, call, raise, fold, and Anteater action. This structure is sent from the client to the server.

GameState:
Stores the synchronized copy of the game state received from the server. Includes player information, community cards, pot value, betting phase, current turn, and dealer position.

NetworkPacket:
Wraps communication data exchanged between the client and server. Used to transfer PlayerAction structures and GameState updates through TCP sockets.

1.2 Client Major Software Components

Client Application (client.c):
Runs the player-side application. Connects to the poker server, sends player actions, receives updated game information, and communicates with the GUI.

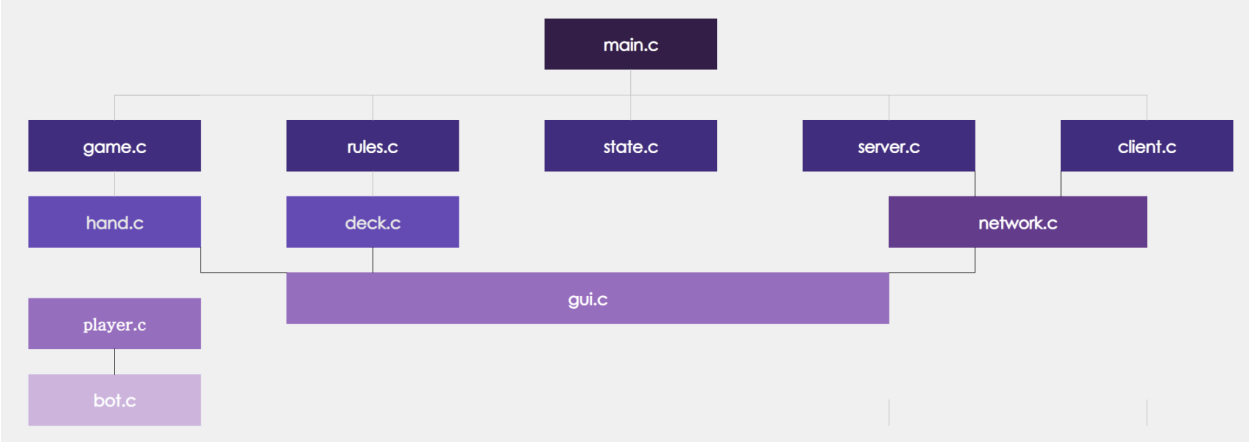
Graphical User Interface (gui.c):
Displays player cards, community cards, betting options, game messages, player banks, and turn information. Also collects player input.

Bot Player Logic (bot.c):
Controls automated computer players. Determines whether the bot should check, call, raise, fold, or use the Anteater action based on the current game state.

Network Communication (network.c):
Handles TCP socket communication between the client and server. Sends and receives NetworkPacket structures and manages socket connections.

Game State Management (state.c):
Maintains the synchronized local copy of the GameState used by the client and GUI.

Diagram of hierarchy



1.3 Client Module Interfaces

Client Application (client.c)
int startClient(const char *serverIP, int port);
void sendPlayerAction(int socketFD, PlayerAction action);
void receiveGameUpdate(int socketFD, GameState *game);
void closeClient(int socketFD);

Graphical User Interface (gui.c)
void initGUI();
void displayGameState(GameState *game);
void displayPlayerHand(Player *player);
void displayCommunityCards(GameState *game);
void displayBettingOptions(GameState *game);
PlayerAction getGUIAction();
void closeGUI();

Bot Player Logic (bot.c)
PlayerAction getBotAction(GameState *game, int playerIndex);
int calculateBotRaiseAmount(GameState *game, int playerIndex);
bool shouldBotFold(GameState *game, int playerIndex);
bool shouldBotUseAnteater(GameState *game, int playerIndex);

Network Communication (communication.c)
int sendMessage(int socketFd, const char *message);
ssize_t receiveMessage(int socketFd, char *buffer, size_t bufferSize);

Game State Management (state.c)
void initGameState(GameState *game);
void resetGameState(GameState *game);
void copyGameState(GameState *destination, GameState *source);
void saveGameState(GameState *game, const char *filename);
void loadGameState(GameState *game, const char *filename);

1.4 Client Overall Program Control Flow

1. The client starts in client.c.
2. The client initializes the GUI and local GameState.
3. The client connects to the server using TCP sockets.
4. The client waits for a synchronized GameState update from the server.
5. The GUI displays player cards, community cards, betting information, and player banks.
6. The current player chooses an action using the GUI or the bot system.
7. The client creates a PlayerAction structure.
8. network.c sends the PlayerAction to the server.
9. The client waits for an updated GameState from the server.
10. The GUI updates after receiving the new GameState.
11. The process repeats until the game ends or the player disconnects.
12. The client closes the connection and shuts down the GUI.

1.5 Automated Client: Poker Bot

The automated client allows the game to include computer-controlled players when there are not enough human players connected. The bot uses the same client-side action system as a human player. Instead of receiving button input from the GUI, bot.c examines the current synchronized GameState and creates a PlayerAction structure.

The bot decision process checks the player's hand strength, the current bet, the player's bank balance, and the current betting phase. If the call amount is too high compared to the bot's bank, the bot may fold. If the bot has a strong hand, it may call, raise, or use the Anteater action. If the bot has a weak hand, it may check when possible or fold when required to call. The bot can also randomly bluff by raising with a weaker hand.

The bot does not directly change the official game state. It only sends a PlayerAction to the server. The server then validates the action using rules.c before applying it to the official GameState.

Important bot functions:

```
PlayerAction getBotAction(GameState *game, int playerIndex);
int calculateBotRaiseAmount(GameState *game, int playerIndex);
bool shouldBotFold(GameState *game, int playerIndex);
bool shouldBotUseAnteater(GameState *game, int playerIndex);
```

2. Poker Server Software Architecture Overview

2.1 Server Main Data Types and Structures

The server stores the authoritative version of the poker game. This means the server keeps the official GameState, validates all player actions, updates the pot and player banks, deals cards, determines winners, and sends synchronized game updates to all clients.

Important server-side structures include Deck, Player, GameState, HandRank, PlayerAction, and NetworkPacket.

Deck stores the official card deck, including all normal cards and the Anteater card. The player stores each player's bank, current bet, hand cards, fold status, seat position, and connection state. GameState stores the complete official game state, including the deck, players, community cards, pot, betting phase, current turn, dealer position, and Anteater effects. HandRank stores the ranking of each poker hand. NetworkPacket stores messages exchanged between the server and clients.

2.2 Server Major Software Components

Server Application (server.c):
The main server module. It opens the server socket, accepts client connections, receives player actions, calls the rules engine, updates the official GameState, and broadcasts updates.

Game Flow Management (game.c):
Controls the round structure. It starts new rounds, deals hole cards, deals the flop, turn, and river, advances betting phases, checks if the round is over, and ends the round.

Rules Engine (rules.c):
Checks whether player actions are legal. It validates check, call, raise, fold, and Anteater actions before the server changes the GameState.

Hand Evaluation System (hand.c):
Evaluates the remaining players' hands after the final betting phase and decides the winner.

Deck Management (deck.c):
Creates, shuffles, and draws cards from the official deck.

Player Management (player.c):
Initializes players, updates player banks and bets, tracks folded players, and checks whether a player is active.

Network Communication (network.c):
Handles sending and receiving packets through TCP sockets.

2.3 Server Module Interfaces

```c
int startServer(int port);
void acceptClients(int serverSocket);
void handleClientAction(GameState *game, PlayerAction action);
void broadcastGameState(GameState *game);
void closeServer(int serverSocket);

void initGame(GameState *game, int numPlayers);
void startNewRound(GameState *game);
void dealHoleCards(GameState *game);
void dealFlop(GameState *game);
void dealTurn(GameState *game);
void dealRiver(GameState *game);
void advanceTurn(GameState *game);
void advancePhase(GameState *game);
bool isRoundOver(GameState *game);
void finishRound(GameState *game);

bool isValidCheck(GameState *game, int playerIndex);
bool isValidCall(GameState *game, int playerIndex);
bool isValidRaise(GameState *game, int playerIndex, int amount);
bool isValidFold(GameState *game, int playerIndex);
bool isValidAnteaterAction(GameState *game, int playerIndex);

void applyCheck(GameState *game, int playerIndex);
void applyCall(GameState *game, int playerIndex);
void applyRaise(GameState *game, int playerIndex, int amount);
void applyFold(GameState *game, int playerIndex);
void applyAnteaterAction(GameState *game, int playerIndex);
```

2.4 Server Overall Program Control Flow

1. The server starts in server.c.
2. The server opens a TCP socket on the selected port.
3. Clients connect to the server.
4. The server initializes the official GameState.
5. The server starts a new round.
6. The deck is initialized and shuffled.
7. Hole cards are dealt to each player.
8. The server waits for a PlayerAction from the current player.
9. rules.c validates the action.
10. If the action is valid, the server updates the official GameState.
11. If the action is invalid, the server sends an error packet to the client.
12. The server broadcasts the updated GameState to all clients.

13. The server advances turns and betting phases.
14. After the river and final betting phase, hand.c evaluates hands.
15. The winner receives the pot.
16. The server starts the next round or closes if the game ends.

Installation

3.1. System requirements and compatibility

Before installing and running the program please make sure your system has and meets the following requirements:

- Operating System (Linux)
- Compiler (gcc)
- Disk Space
- UCI EECS lab machines

3.2 Setup and configuration

In order to install the program, please follow the steps below:

Step 1: Extract Files

a. To extract the files, type the following into your terminal:
"gtar xvzf Poker_V1.0_src.tar.gz"

Step 2: Change the Directory:

a. After you have finished extracting the files, change into the program directory. To change into the program directory, type the following into your terminal:
"cd poker"

Step 3: Compile the Program

a. Once you have changed to the correct directory, compile the program by typing:
"make"

Step 4: Run the Server

a. In the first terminal window, start the poker server by typing:
"make run-server"

b. The server will wait for client connections.

Step 5: Run the Client

a. In another terminal window, connect a client to the server by typing:
"make run-client"

b. The client will connect to the server and begin the poker session.

Step 6: Clean Compiled Files

a. To remove compiled files and executables, type:
"make clean"

3.3. Building, compilation, and installation

After the project has been extracted and the developer has navigated to the poker directory. The software, Makefile, will automate the compilation and link of all source files into a single

executable. The make command will compile the program, generate the necessary object files, and produce the final executable in the output directory. To remove compiled files and reset the project directory, the make clean command can be used. No separate installation process is needed after the files are compiled. Once the build process is complete, the executable can be run directly from the project directory.

4.1 Description of Data Structures

The main data structures used by Anteater Poker are Card, Player, Deck, GameState, PlayerAction, and NetworkPacket. These structures are shared between modules so the server, client, rules engine, GUI, and bot can communicate using the same data format.

ENUMERATED TYPES

```
typedef enum {
     HUMAN_PLAYER,
     BOT_PLAYER
} PlayerType;
```

```
typedef enum {
      ANTEATER = 0,
      ACE= 1,
      TWO= 2,
      THREE= 3,
      FOUR = 4,
      FIVE = 5,
      SIX = 6,
      SEVEN = 7,
      EIGHT = 8,
      NINE = 9,
      TEN = 10,
      JACK = 11,
      QUEEN = 12,
      KING = 13
} Rank;
```

Card Type

This is an enum to decide whether a card is an anteater card or a normal card

```
typedef enum {
      NORMAL_CARD,
      ANTEATER_CARD
} CardType;
```

DECK STRUCTURE

Each of our card's suits will be represented by an enum named suit. Each card, excluding the anteater, will be assigned to a suit.

```
typedef enum {
    HEARTS,
    DIAMONDS,
    CLUBS,
    SPADES
} Suit;
```

The Card structure represents one card in the game. It stores the rank, suit, and whether the card is a normal card or the Anteater card.

```
typedef struct {
    Rank rank;
    Suit suit;
    CardType type;
} Card;
```

The Player structure stores information about a player including their name, bank balance, current bet, hand cards, fold status, and whether the player is human or bot-controlled.

```
typedef struct {
    char name[32];
    int bank;
    int currentBet;
    Card hand[2];
    bool folded;
    PlayerType type;
} Player;
```

The Deck structure stores all cards currently used in the game. The top variable keeps track of the next card to draw.

```
typedef struct {
    Card cards[53];
    int top;
} Deck;
```

The GameState structure stores the current state of the poker game including all players, the deck, community cards, pot value, and turn information.

```
typedef struct {
    Player players[MAX_PLAYERS];
    Deck deck;
    Card communityCards[5];
    int pot;
    int currentPlayer;
    int dealerPosition;
} GameState;
```

The PlayerAction structure stores a player move sent from the client to the server.

```
typedef struct {
    int playerIndex;
    int actionType;
    int amount;
} PlayerAction;
```

The NetworkPacket structure is used to transfer information between the client and server through TCP sockets.

```
typedef struct {
    int packetType;
    PlayerAction action;
    GameState game;
} NetworkPacket;
```

4.2 Detailed Description of Functions and Parameters

Function 1: Deal
void deal (player, deck)

- Takes in which player and the next two cards from the deck
- **Output**: Assign the cards from the deck to the user

Function 2: set small/big blind
void set blind (player)

- Assigns a player to small blind
- Moves the pointer to the next player
- Forces the player to put in small blind from their bank

Function 3: check/call
void check (player, bank)

- checks to see if the player's bet equals the round's bet
- Changes to call if the player's bet is less than the round's bet
- **Output:** moves the check pointer to check the next player

Function 4: Pot
void pot (player, bank)

- checks to see if the player's bet equals the round's bet
- Changes to call if the player's bet is less than the round's bet
- **Output:** moves the check pointer to check the next player

Function 5: Hand
void hand (player, deck, board)

- Adds a copy of the board to the hand
- Evaluates which player has the most value
- **Output:** decides winner

Function 6: check full house
Void check_fullhouse (player, hand, board)

- Check each card rank and see if they match the requirement for the full house
- **Output:** increase the player's win count

Function 7: Server
make run-server

- in terminal
- Can do make run-client to join
- **Output:** creates server so others can join remotely

Function 7: Close Server
lsof -i

- in terminal
- **Output:** closes the server so it can be reran

4.3 Detailed Description of Communication Protocol

Anteater Poker uses TCP socket communication between the client and server. The client sends player actions to the server using NetworkPacket structures. The server validates each action, updates the official GameState, and broadcasts the updated GameState to all connected clients.

Communication function calls:

int createSocket();
Creates a TCP socket.

int connectToServer(const char *serverIP, int port);
Connects the client to the server using the server IP address and port number.

int sendPacket(int socketFD, NetworkPacket *packet);
Sends a NetworkPacket through the socket.

int receivePacket(int socketFD, NetworkPacket *packet);
Receives a NetworkPacket from the socket.

void closeConnection(int socketFD);
Closes the client or server socket connection.

Communication packet types:

PACKET_CONNECT:
Sent when a client first connects to the server.

PACKET_ACTION:
Sent from the client to the server when a player chooses check, call, raise, fold, or Anteater action.

PACKET_GAME_STATE:
Sent from the server to all clients after the official GameState changes.

PACKET_ERROR:
Sent from the server to a client when the client sends an invalid action.

PACKET_DISCONNECT:
Sent when a client leaves the game or the server closes the connection.

5.1 Partitioning of Tasks

TASK 1: CORE ENGINE AND MAIN PROGRAM (main.c)

The central controller that coordinates all modules, manages the game loop, and handles turn alternation between the human player and the computer opponent. This module initializes the board, invokes the input handler, dispatches moves to the rules engine, and triggers the display after each turn.

TASK 2: GAME FLOW MANAGEMENT (game.c)

This module controls the main poker round logic and connects the core gameplay systems together. It manages the order of play, starts new rounds, handles betting phases, deals community cards, checks when a round should end, and calls the hand evaluation system to determine the winner.

TASK 3: RULES ENGINE (rules.c)

This module enforces the official Texas Hold'em rules and the custom Anteater Poker rules. It checks whether player actions such as call, raise, fold, or special Anteater actions are valid, applies rule changes to the game state, and ensures that the game progresses legally.

TASK 4: HAND EVALUATION SYSTEM (hand.c)

This module determines the strength of each player's poker hand. It compares the player's hole cards with the community cards, detects hands such as pairs, straights, flushes, full houses, and Anteater-specific combinations, then ranks players to decide the winner.

TASK 5: CARD AND DECK MANAGEMENT (deck.c)

This module creates and manages the card deck used in the game. It initializes the deck, shuffles cards randomly, deals cards to players, deals community cards, and tracks which cards have already been used.

TASK 6: PLAYER MANAGEMENT (player.c)

This module stores and updates player information throughout the game. It manages player names, seats, points, cards in hand, current bet, fold status, and whether the player is human, computer-controlled, or disconnected.

TASK 7: CLIENT APPLICATION (client.c)

This module runs the player-side application that connects to the poker server. It sends player actions to the server, receives updated game information, communicates with the GUI, and allows the user to interact with the game remotely.

TASK 8: SERVER APPLICATION (server.c)

This module runs the central host server for the poker game. It accepts client connections, manages connected players, maintains the official game state, coordinates turns, sends updates to all clients, and prevents clients from directly modifying the game unfairly.

TASK 9: NETWORK COMMUNICATION (network.c)

This module handles TCP/IP socket communication between the server and clients. It defines the message format, sends and receives packets, manages connection errors, and allows game actions and state updates to be transferred across the network.

TASK 10: GRAPHICAL USER INTERFACE (gui.c)

This module displays the poker game visually to the user. It shows the player's cards, community cards, player names, points, betting options, game messages, and server-side control information, while also collecting button clicks or input from the user.

TASK 11: BOT PLAYER LOGIC (bot.c)

This module controls the computer opponent or automated player. It evaluates the current game situation, decides whether to call, raise, fold, or use an Anteater special action, and allows empty seats to be filled by non-human players.

TASK 12: GAME STATE MANAGEMENT (state.c)

This module stores the current condition of the poker game. It tracks the deck, players, pot, community cards, current round, current turn, dealer position, active players, and any Anteater rule effects that are currently active.

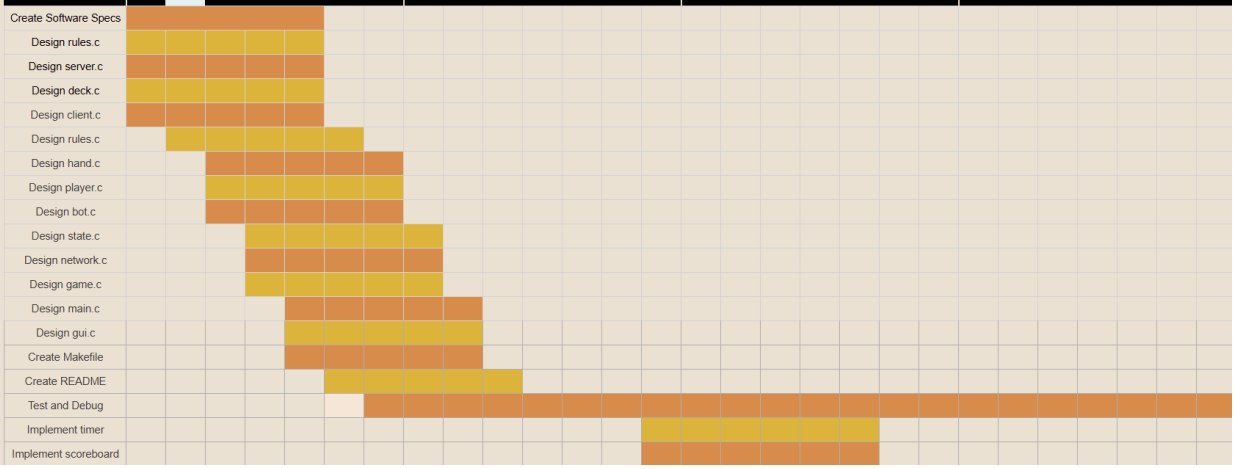
5.2 Team Member Responsibilities

TEAM MEMBER	MAIN RESPONSIBILITIES	SHARED OR SECONDARY RESPONSIBILITIES
QUEENCY TAN	Graphic user interface (gui.c), Rules (rules.c), server (server.c)	core engine (main.c), game flow (game.c)
MIA NESS	Game State (game_state.c), Game Flow (game.c), deck (deck.c)	Client (client.c), server (server.c), network (network.c)
AUSTIN NGUYEN	Hand evaluation (hand.c), deck (deck.c), Rules (rules.c), player (player.c)	core engine (main.c), game flow (game.c)
NAVEEN KHAN	Bot player (bot.c), Game flow (game.c), core engine (main.c)	Client (client.c), server (server.c), network (network.c), makefile
AMAN MACWAN	Client (client.c), server (server.c), network (network.c)	Core engine (main.c), game flow (game.c), rules (rules.c)
GURVIN DHANJAL	Client (client.c), server (server.c), network (network.c)	core engine (main.c), makefile, deck (deck.c)

WHILE SPECIFIC MODULES HAVE BEEN ASSIGNED TO INDIVIDUAL MEMBERS, ALL MEMBERS WILL MAKE CONTRIBUTIONS TO THE CREATION OF THE FINAL PRODUCT.

Four week project timeline

Start Date: 5/11/2026



6. COPYRIGHT

Anteater Poker is an original adaptation of the traditional poker game, developed exclusively for educational purposes at the University of California, Irvine. The Anteater card and associated rules are defined by the EECS 22L course specification authored by Prof. Rainer Doemer.

7. REFERENCES

Doemer, R. “EECS 22L Project 2: Anteater Poker” University of California, Irvine, May 4, 2026.

Texas holdem poker rules.

https://bicyclecards.com/how-to-play/texas-holdem-poker

GCC, the GNU Compiler Collection. Free Software Foundation.

https://gcc.gnu.org/

GNU Make Manual. Free Software Foundation.
https://www.gnu.org/software/make/manual/

Git — Distributed Version Control System. https://git-scm.com/

8. INDEX